SIMULATORE DI MECCANISMI

Manuale utente completo

Versione del simulatore: V. 0.4

Data: 9 novembre 2026

Come scrivere un file JSON utilizzando i tipi di oggetti
definiti nella cartella `types/` del repository

https://github.com/jumpjack/mechsim

Questo documento descrive in dettaglio ogni sezione del file MACHINE,
ogni tipo di ingranaggio, link e parte, con parametri, limiti,
esempi funzionanti e note per lo sviluppatore.

Indice

1. Introduzione e struttura del file MACHINE
2. Sezioni macro del file JSON
2.1 title, show, groups
2.2 consts (costanti e espressioni)
2.3 variants (varianti selezionabili con checkbox)
2.4 controls (slider e controlli geometrici)
2.5 inputs (ingressi cinematici)
2.6 axes (assi)
2.7 gears (ingranaggi)
2.8 links (vincoli e collegamenti)
3. Tipi di ingranaggi (gears)
3.1 spur — ruota cilindrica a denti dritti
3.2 bevel — ruota conica
3.3 worm — vite senza fine
3.4 wormwheel — ruota elicoidale per vite
3.5 maltese — croce di Malta
3.6 geneva_driver — ruota porta-perno (Geneva)
3.7 cylinder — cilindro semplice
3.8 anchor / anchor_CS — ancora (scappamento)
3.9 escapement / escape_wheel_CS — ruota di scappamento
4. Tipi di link (links)
4.1 ratchet — nottolino
4.2 brake — freno
4.3 rope — fune con peso
4.4 hooke — snodo di Hooke
4.5 pin_slot — perno in asola
4.6 rail — binario / guida
4.7 geneva — meccanismo a croce di Malta
4.8 anchor_escapement / graham_escapement_CS
5. Tipi di parti degli assi (parts)
5.1 cylinder, box, prism
6. Proprietà indirizzabili nelle espressioni (.pr, .z, .m, .ox, .oy, .oz, …)
7. Posizione assiale dell'ingranaggio: w oppure pos + thickness
8. Foro centrale, dente di fissaggio (profilo a D) e asse abbinato
9. Marker degli ingranaggi: cosa sono, a cosa servono, come si specificano
10. Note generali sulle espressioni e formule
11. Note per lo sviluppatore (riepilogo)

1. Introduzione e struttura del file MACHINE

Il simulatore legge un oggetto JavaScript chiamato `MACHINE` (può essere un file `.js` con `var MACHINE = { … };` oppure un JSON puro). Il file può contenere commenti di riga (`//`) e di blocco (`/* … */`) e virgole finali: il caricatore li rimuove prima del parsing. Le stringhe devono usare doppi apici.

Le tre famiglie di oggetti sono organizzate in cartelle sotto `js/types/`:

- **gears/** — ingranaggi e organi rotanti (type nel JSON di un elemento di MACHINE.gears)
- **links/** — vincoli cinematici e collegamenti speciali (type in MACHINE.links)
- **parts/** — forme geometriche degli assi (shape in axes[].parts)

Ogni tipo di gear o link è composto da due file che si fondono automaticamente: `create.js` (geometria, mesh Babylon, controlli UI) e `solve.js` (validazione JSON, parametri, cinematica). Quando la macchina usa un `"type": "nome"`, il loader carica da solo i file corrispondenti.

I valori numerici possono quasi sempre essere scritti come **espressioni** (stringhe) che il motore valuta usando le costanti, gli slider, i raggi e i denti degli ingranaggi già risolti. Esempio: `"module": "mR"`, `"w": ["yA0", "yA0+face"]`.

2. Sezioni macro del file JSON

2.1 title, show, groups

title (stringa, opzionale): titolo visualizzato nel pannello laterale.

show (array, opzionale): elenco di elementi da evidenziare nella tabella delle velocità. Ogni elemento è `{"gear": "id", "label": "…"}` oppure `{"axis": "id", "label": "…"}`.

groups (array, opzionale): definisce i gruppi di visibilità controllabili con le checkbox nel pannello. Esempio: `[{"id": "upper", "label": "Parte superiore", "default": true}, …]`. Gli assi e gli oggetti possono avere il campo `"group": "upper"`.

2.2 consts (costanti e espressioni)

Oggetto chiave-valore. I valori possono essere numeri o stringhe-espressione. Le costanti sono valutate in ordine e possono riferirsi a quelle precedenti e agli ingranaggi (es. `"e": "gears.gear_P.pr + gears.gear_Q.pr"`). Sono la base per parametrizzare tutta la macchina in modo coerente.

```
"consts": {
  "module": 1,
  "face": 8,
  "gap": 7,
  "yA0": "yW1 + drumLen",
  "e": "gears.gear_P.pr + gears.gear_Q.pr"
}
```

2.3 variants — varianti selezionabili con checkbox

La sezione `variants` permette di definire configurazioni alternative della macchina (geometria diversa, assi diversi, ingranaggi diversi) che l'utente seleziona tramite checkbox o radio nel pannello. Solo gli elementi la cui variante è attiva vengono considerati dal validatore e dal risolutore.

Struttura:

```
"variants": {
  "input": {
    "label": "Ingresso dell'asse c",
    "active": "bevel",
    "options": {
      "spur": "Ruote gemelle R–R′ e asse a",
      "bevel": "Coppia conica R–R′ con asta lunga e snodo S"
    }
  }
```

```
}
```

• **label**: testo mostrato accanto al selettore.
• **active**: valore iniziale (deve essere una delle chiavi di options).
• **options**: mappa valore → etichetta visualizzata.

Negli elementi di axes, gears, links, controls, inputs si può scrivere `"variant": {"input": "bevel"}`. L'elemento è attivo solo se la variante corrente coincide. Si possono combinare più chiavi di variante (AND logico).

I controlli possono avere ranges dipendenti dalla variante:

```
{"id": "zR", "label": "Denti di R", "variantKey": "input",
 "ranges": {
   "spur":  {"min": 50, "max": 100, "value": 60},
   "bevel": {"min": 20, "max": 60,  "value": 30}
 }}
```

2.4 controls — slider e controlli geometrici

Array di oggetti che generano slider nel pannello «Geometria». I valori sono disponibili nelle espressioni con il loro id (es. `latitude`).

Campi principali:

`id` [OBBLIGATORIO] · solo valore letterale
Identificatore univoco; usato nelle formule.

`label` [opzionale] · solo valore letterale
Testo mostrato all'utente.

`step` [opzionale] · solo valore letterale
Incremento dello slider (default 1).

`ranges` [OBBLIGATORIO] · solo valore letterale
Oggetto con chiavi di variante (o "all") e sotto-oggetti {min, max, value}.

`variantKey` [opzionale] · solo valore letterale
Se presente, lo slider mostra solo i ranges della variante attiva.

`hidden` [opzionale] · solo valore letterale
Se true, lo slider non è mostrato ma il valore è comunque disponibile.

```
"controls": [
  {"id": "latitude", "label": "Latitudine (°)", "step": 1,
   "ranges": {"all": {"min": -90, "max": 90, "value": 42}}},
  {"id": "zR", "label": "Denti di R", "variantKey": "input", "step": 1,
   "ranges": {
     "spur":  {"min": 50, "max": 100, "value": 60},
     "bevel": {"min": 20, "max": 60,  "value": 30}
   }}
]
```

2.5 inputs — ingressi cinematici

Definisce quali assi sono «motori» (fonti di moto). Nel pannello appare un selettore di velocità (giri/min) per l'ingresso selezionato. Un input tipico:

```
"inputs": [
  {"id": "motore", "axis": "axis_A", "label": "Motore a contrappeso",
   "variant": {"input": "spur"}}
]
```

• **id**: identificatore.
• **axis**: id dell'asse che riceve la velocità (obbligatorio, deve esistere).
• **label**: etichetta UI.
• **variant**: (opzionale) rende l'input attivo solo in certe varianti.

2.6 axes — definizione degli assi

Array di oggetti che definiscono gli assi di rotazione. Ogni asse ha un sistema di riferimento (origine O, direzione dir, riferimento ref) e può contenere parti geometriche.

Parametri principali di un asse:

`id` [OBBLIGATORIO] · solo valore letterale
Identificatore univoco (obbligatorio).

`name` [opzionale] · solo valore letterale
Nome descrittivo per UI/log.

`dir` [opzionale] · può essere formula
Versore direzione dell'asse [x,y,z] (obbligatorio se non derivato da place).

`ref` [opzionale] · può essere formula
Versore di riferimento per l'angolo zero [x,y,z].

`view / viewLabel` [opzionale] · può essere formula
Direzione e etichetta per la vista ortogonale.

`color` [opzionale] · solo valore letterale
Colore dell'asse (es. "#9aa4ae" o nome).

`group` [opzionale] · solo valore letterale
Id del gruppo di visibilità.

`place` [opzionale] · può essere formula
Posizionamento: at, coaxial, offset, byMesh, byApex (vedi sotto).

`parts` [opzionale] · solo valore letterale
Array di parti geometriche (cylinder, box, prism).

`carrier` [opzionale] · solo valore letterale
Asse portante (per satelliti).

`rotate` [opzionale] · può essere formula
Rotazioni iniziali: [{"around": "self"|id, "deg": expr}].

`through` [opzionale] · può essere formula
Vincolo di passaggio (punto o asse).

`refFrom` [opzionale] · solo valore letterale
Copia il riferimento da un altro asse (con through).

`perpendicularTo` [opzionale] · può essere formula
Vincolo di perpendicolarità.

`spinPerpTo` [opzionale] · può essere formula
Spin perpendicolare.

`variant` [opzionale] · solo valore letterale
Attivazione condizionale per variante.

`compare` [opzionale] · solo valore letterale
Lista di assi con cui confrontare le velocità nella tabella.

Modalità di place:

• **at**: punto assoluto [x,y,z].
• **coaxial**: id di un altro asse (stessa retta).
• **offset**: {"from": id|punto, "by": [dx,dy,dz]}.
• **byMesh**: posiziona usando l'interasse dell'ingranaggio indicato (richiede dir).
• **byApex**: per ruote coniche, allinea all'apice dell'ingranaggio.

2.7 gears — ingranaggi

Array di oggetti. Il campo `type` (default "spur") seleziona il tipo nella cartella types/gears/. Parametri comuni a quasi tutti i tipi:

`id` [OBBLIGATORIO] · solo valore letterale
Identificatore univoco (obbligatorio).

`type` [opzionale] · solo valore letterale
Tipo di ingranaggio (default "spur").

`axis` [OBBLIGATORIO] · solo valore letterale

Id dell'asse su cui è montato (obbligatorio).

`teeth / z` [OBBLIGATORIO] · può essere formula · usabile come input/control
Numero di denti (obbligatorio per la maggior parte dei tipi).

`module / m` [OBBLIGATORIO] · può essere formula · usabile come input/control
Modulo in mm (obbligatorio per molti tipi).

`w` [opzionale] · può essere formula
Intervallo assiale [w0, w1] lungo l'asse. Non e' l'unica possibilita': in alternativa si usano insieme pos e thickness (vedi sezione 7).

`pos + thickness` [opzionale] · può essere formula
Alternativa a w: pos = coordinata della faccia piu' vicina all'origine dell'asse; thickness = spessore (positivo verso +asse). Vanno indicati entrambi; se c'e' anche w esplicito, vince w.

`meshWith` [opzionale] · solo valore letterale
Id dell'ingranaggio con cui e' in presa.

`color` [opzionale] · solo valore letterale
Colore della mesh.

`fixed` [opzionale] · solo valore letterale
Se true (default) l'ingranaggio e' solidale all'asse. Se false, oppure free:true, e' libero di ruotare sull'asse (asse virtuale).

`free` [opzionale] · solo valore letterale
Se true crea un asse virtuale coassiale e l'ingranaggio non e' trascinato dall'asse ospite.

`bore` [opzionale] · può essere formula
Foro centrale: numero/espressione (raggio esplicito), false (nessun foro), oppure omesso (calcolo automatico). Vedi sezione 8.

`flat / flatAngle` [opzionale] · può essere formula
Per il foro a D: flat = frazione 0-1 del raggio (default 0.8); flatAngle = orientamento in gradi (default 0). Vedi sezione 8.

`marker` [opzionale] · può essere formula
Marker (sfera): true (default), false, numero (raggio mm), oppure {radius, color, at}. Vedi sezione 9.

`angle / phase` [opzionale] · può essere formula
Rotazione iniziale in gradi.

`variant` [opzionale] · solo valore letterale
Attivazione condizionale.

`group` [opzionale] · solo valore letterale
Gruppo di visibilita'.

2.8 links — vincoli e collegamenti

Array di oggetti con campo `type` obbligatorio. Ogni tipo ha i propri parametri (vedi sezioni dedicate). Possono avere `variant` e `active` (boolean o formula).

3. Tipi di ingranaggi (gears)

3.1 spur — ruota cilindrica a denti dritti

Tipo di default. Ingranaggio cilindrico a denti diritti. Può ingranare solo con altre ruote spur. Il raggio primitivo è z·m/2. Supporta settori di denti (degStart/degEnd), foro, sfacciatura e fase iniziale.

Parametri specifici:

`teeth` [OBBLIGATORIO] · può essere formula · usabile come input/control
Numero di denti (≥ 3).

`module` [OBBLIGATORIO] · può essere formula · usabile come input/control
Modulo (mm).

`w` [OBBLIGATORIO] · può essere formula · usabile come input/control
[w0, w1] posizione assiale.

`degStart / degEnd` [opzionale] · può essere formula · usabile come input/control
Settore di denti in gradi (opzionale).

`bore` [opzionale] · può essere formula · usabile come input/control
Foro: numero, false o {r:…}.

`flat / flatAngle` [opzionale] · può essere formula · usabile come input/control
Sfacciatura (0–1) e angolo.

`angle / phase` [opzionale] · può essere formula · usabile come input/control
Rotazione iniziale (gradi).

Esempio completo funzionante:

```
{
  "title": "Esempio spur semplice",
  "consts": { "m": 2, "face": 10 },
  "axes": [
    { "id": "ax1", "dir": [0,1,0], "ref": [1,0,0],
      "place": { "at": [0,0,0] },
      "parts": [ { "shape": "cylinder", "r": 3, "w": [-5, 50] } ] },
    { "id": "ax2", "dir": [0,1,0], "ref": [1,0,0],
      "place": { "byMesh": "g1", "dir": [1,0,0] },
      "parts": [ { "shape": "cylinder", "r": 3, "w": [-5, 50] } ] }
  ],
  "gears": [
    { "id": "g1", "type": "spur", "axis": "ax1",
      "teeth": 20, "module": "m", "w": [0, "face"], "color": "#4a90d9" },
    { "id": "g2", "type": "spur", "axis": "ax2",
      "teeth": 30, "module": "m", "w": [0, "face"],
      "meshWith": "g1", "color": "#d94a4a" }
  ],
  "inputs": [ { "id": "in", "axis": "ax1", "label": "Motore" } ],
  "groups": [ { "id": "main", "label": "Tutto", "default": true } ]
}
```

Nota sviluppatore: il tipo spur è autosufficiente in create.js + solve.js. Esiste un piccolo hook in core/09-resolve.js per angle/phase; non è necessario toccare altro per modifiche ordinarie.

3.2 bevel — ruota conica

Ruota dentata conica. Richiede apexW (posizione dell'apice lungo l'asse) e di solito un partner con meshWith. Il semiangolo δ può essere calcolato dalla coppia o fornito con delta.

`teeth` [OBBLIGATORIO] · può essere formula · usabile come input/control
Numero di denti.

`module` [OBBLIGATORIO] · può essere formula · usabile come input/control
Modulo (mm).

`apexW` [OBBLIGATORIO] · può essere formula · usabile come input/control
Posizione dell'apice lungo l'asse (espressione o numero).

`side` [opzionale] · può essere formula · usabile come input/control

1 o -1 (verso del cono rispetto a dir dell'asse).

`delta` [opzionale] · può essere formula · usabile come input/control
Semiangolo del cono in gradi (se non c'è partner).

`w` [opzionale] · può essere formula · usabile come input/control
Intervallo assiale (può essere derivato).

Esempio minimo:

```
{
  "title": "Coppia conica",
  "consts": { "m": 2 },
  "axes": [
    { "id": "axH", "dir": [0,1,0], "ref": [1,0,0], "place": {"at":[0,0,0]},
      "parts": [{"shape":"cylinder","r":3,"w":[-20,40]}] },
    { "id": "axV", "dir": [1,0,0], "ref": [0,1,0],
      "place": {"byApex":"b1"}, "parts": [{"shape":"cylinder","r":3,"w":[-20,40]}] }
  ],
  "gears": [
    { "id": "b1", "type": "bevel", "axis": "axH",
      "teeth": 24, "module": "m", "apexW": 0, "side": 1, "w": [0, 12],
      "color": "#4a90d9" },
    { "id": "b2", "type": "bevel", "axis": "axV",
      "teeth": 24, "module": "m", "apexW": 0, "side": 1, "w": [0, 12],
      "meshWith": "b1", "color": "#d94a4a" }
  ],
  "inputs": [{"id":"in","axis":"axH","label":"Ingresso"}]
}
```

Nota sviluppatore: create.js e solve.js contengono tutta la logica specifica. Il calcolo di δ e del contatto è nel tipo; il nucleo gestisce solo la presa generica.

3.3 worm — vite senza fine

Vite senza fine (cilindrica a filetto elicoidale). Di solito si accoppia con wormwheel. Il parametro q è il numero di principi (filetti). Il raggio è tipicamente q·m/2.

`q` [OBBLIGATORIO] · può essere formula · usabile come input/control
Numero di principi (filetti) della vite.

`module` [OBBLIGATORIO] · può essere formula · usabile come input/control
Modulo (mm).

`w` [OBBLIGATORIO] · può essere formula · usabile come input/control
Intervallo assiale della vite.

`leadAngle / gamma` [opzionale] · può essere formula · usabile come input/control
Angolo d'elica (se non derivato).

Nota sviluppatore: la formula del raggio q·m/2 è ora dentro il tipo worm (non più globale). Modifiche tipiche restano in create.js/solve.js. Attenzione alla direzione di meshWith (meglio dichiararlo sulla ruota).

3.4 wormwheel — ruota elicoidale per vite

Ruota a denti elicoidali o a gola progettata per ingranare con una vite. Parametri principali: teeth, module, w, e di solito meshWith verso la vite.

Nota sviluppatore: contenuto principalmente in create.js e solve.js del tipo. Il contatto speciale vite-ruota è gestito nel tipo (contact / meshRow).

3.5 maltese — croce di Malta

Croce di Malta (Geneva driven wheel). Non ha denti tradizionali: usa slots (numero di scanalature). Il raggio può essere fornito o calcolato. Si usa insieme al link geneva e al gear geneva_driver.

`slots` [OBBLIGATORIO] · può essere formula · usabile come input/control
Numero di scanalature (tipicamente 4).

`w` [OBBLIGATORIO] · può essere formula · usabile come input/control
Spessore assiale [w0,w1].

`r` [opzionale] · può essere formula · usabile come input/control
Raggio esterno (opzionale, altrimenti calcolato).

Nota sviluppatore: noRadiusCheck è attivo. La cinematica vera e propria è nel link geneva; il gear fornisce solo la geometria. Modifiche geometriche in create.js del maltese.

3.6 geneva_driver — ruota porta-perno

Disco o ruota che porta il perno per la croce di Malta. Parametri tipici: w, pinSide, pinDiameter, lockGap, r. La posizione angolare del perno si imposta nel link geneva (pinAngle).

Nota sviluppatore: self-contained in create.js/solve.js. Il raggio reale viene spesso aggiornato dal link.

3.7 cylinder — cilindro semplice

Organo cilindrico senza denti (tamburo, rullo, ecc.). Richiede r e w. Utile come puleggia o massa rotante.

`r` [OBBLIGATORIO] · può essere formula · usabile come input/control
Raggio (mm).

`w` [OBBLIGATORIO] · può essere formula · usabile come input/control
[w0, w1].

Nota sviluppatore: tipo minimalista, tutto in create.js + solve.js.

3.8 anchor / anchor_CS — ancora di scappamento

Ancora (pallet) per scappamento. anchor è la versione classica; anchor_CS è la versione «Clock Simulator» con parametri più ricchi (pendolo, bob, bracci). Si usa tipicamente con i link di scappamento.

Parametri tipici di anchor_CS:

`pendulumLength` [OBBLIGATORIO] · può essere formula · usabile come input/control
Lunghezza del pendolo (mm).

`w` [OBBLIGATORIO] · può essere formula · usabile come input/control
Intervallo assiale.

`bobRadius / bobThickness` [opzionale] · può essere formula · usabile come input/control
Dimensioni del bilanciere.

`rodWidth / armWidth / armLength / armAngle` [opzionale] · può essere formula · usabile come input/control
Geometria dei bracci.

`hubRadius / rodGap / armConnector` [opzionale] · può essere formula · usabile come input/control
Dettagli costruttivi.

Nota sviluppatore: i file create.js/solve.js di anchor e anchor_CS contengono la logica. Per lo scappamento completo servono anche i link corrispondenti. Verificare eventuali riferimenti sparsi nei file di scappamento più vecchi.

3.9 escapement / escape_wheel_CS — ruota di scappamento

Ruota di scappamento. escape_wheel_CS offre parametri di profilo dente (toothDepth, hook, valley, rimWidth, spokes, hubRadius).

`r` [OBBLIGATORIO] · può essere formula · usabile come input/control
Raggio delle punte dei denti.

`teeth` [OBBLIGATORIO] · può essere formula · usabile come input/control
Numero di denti.

`w` [OBBLIGATORIO] · può essere formula · usabile come input/control
Spessore assiale.

`toothDepth / hook / valley / rimWidth / spokes / hubRadius` [opzionale] · può essere formula · usabile come input/control
Parametri di profilo (opzionali, con default).

Nota sviluppatore: self-contained. La cinematica di impulso/blocco è nel link di scappamento associato.

4. Tipi di link (links)

4.1 ratchet — nottolino

Nottolino unidirezionale: permette la rotazione relativa solo in un senso. Quando il senso «proibito» viene rilevato, il link blocca le due velocità angolari.

`driver` [OBBLIGATORIO] · solo valore letterale · usabile come input/control
Id asse motore.

`driven` [OBBLIGATORIO] · solo valore letterale · usabile come input/control
Id asse condotto.

`sign` [opzionale] · solo valore letterale · usabile come input/control
Senso consentito (+1 / -1, default 1).

`messages` [opzionale] · solo valore letterale · usabile come input/control
Oggetto con testi: drives, winding, freewheel, blocks.

```
{ "type": "ratchet", "driver": "axis_A", "driven": "axis_B", "sign": 1,
  "messages": {
    "drives": "Il nottolino trascina",
    "winding": "Caricamento",
    "freewheel": "Ruota libera",
    "blocks": "Bloccato dal nottolino"
  }
}
```

Nota sviluppatore: interamente in create.js (se presente) + solve.js. Usa i ganci lockable/lock/constraints/status del registro. Nessun codice sparso critico.

4.2 brake — freno

Freno che può bloccare un asse. Di solito ha un controllo UI (checkbox o stato) e un asse di riferimento.

Nota sviluppatore: create.js + solve.js. Il pannello freni è gestito dal tipo (panel).

4.3 rope — fune con peso

Fune avvolta su un tamburo (asse) con un peso all'estremità. Non impone vincoli di velocità, ma la lunghezza dipende dall'angolo dell'asse. Parametri: axis, at (punto di attacco), radius, len0, min, max.

`axis` [OBBLIGATORIO] · può essere formula · usabile come input/control
Asse del tamburo.

`at` [OBBLIGATORIO] · può essere formula · usabile come input/control
Punto 3D di uscita della fune.

`radius` [OBBLIGATORIO] · può essere formula · usabile come input/control
Raggio di avvolgimento.

`len0` [OBBLIGATORIO] · può essere formula · usabile come input/control
Lunghezza iniziale della fune libera.

`min / max` [OBBLIGATORIO] · può essere formula · usabile come input/control
Limiti di lunghezza.

Nota sviluppatore: create.js costruisce i mesh di fune e peso; solve.js valida e risolve. Autocontenuto.

4.4 hooke — snodo di Hooke (giunto cardanico)

Giunto di Hooke / cardanico tra due assi non necessariamente allineati. Parametri tipici: i due assi e le dimensioni dei bracci della crociera.

Nota sviluppatore: create.js contiene createHooke. Nel README è segnalato un bug storico (h.def.arm finisce nel parametro sbagliato); le dimensioni della crociera restano ai default se non corretti. La logica è nei due file del tipo, ma conviene verificare i punti di chiamata.

4.5 pin_slot — perno in asola

Vincolo di perno che scorre in un'asola. Definisce la relazione geometrica tra due assi/punti.

Nota sviluppatore: create.js + solve.js. Nessuna evidenza di codice sparso critico.

4.6 rail — binario / guida

Guida lineare / binario. Può avere marker. Usato per vincoli di scorrimento.

Nota sviluppatore: self-contained. create.js gestisce mesh e marker.

4.7 geneva — meccanismo a croce di Malta

Link che realizza la cinematica della croce di Malta collegando un geneva_driver (o spur usato come driver) e una maltese. Parametri tipici: driver, driven, pinAngle, interasse, ecc.

Nota sviluppatore: la parte più grossa è in solve.js (vincoli, fasi di moto). create.js può essere quasi vuoto. È uno dei tipi più complessi; modifiche richiedono attenzione all'ordine delle fasi (order).

4.8 anchor_escapement / graham_escapement_CS

Link di scappamento che coordinano ancora e ruota di scappamento (impulso e blocco). graham_escapement_CS è la versione più recente e completa.

Nota sviluppatore: solve.js molto corposo. Verificare se restano riferimenti residui in core o in altri file di scappamento più vecchi (anchor_escapement vs graham).

5. Tipi di parti degli assi (parts)

Le parti sono definite dentro axes[].parts e servono solo alla geometria visuale e all'export STL. Non hanno cinematica propria. Ogni parte ha shape e parametri specifici; può avere color, at (offset 2D nel piano dell'asse), n (suddivisioni).

5.1 cylinder

Cilindro (pieno o cavo). Parametri: r (obbligatorio), w [w0,w1] (obbligatorio), n, flat, hollow (raggio interno).

```
{"shape": "cylinder", "r": 3.5, "w": [0, 110], "n": 20}
{"shape": "cylinder", "r": 6, "w": [10, 40], "hollow": 4, "color": "#333"}
```

Nota sviluppatore: solo create.js (solid + validate). Nessun solve.js.

5.2 box

Parallelepipedo. Parametri: w [w0,w1] (lungo l'asse), half [hx, hy] (semi-dimensioni trasversali), at opzionale.

```
{"shape": "box", "w": [20, 22], "half": [0.5, 12], "color": "#0000ff"}
```

Nota sviluppatore: solo create.js.

5.3 prism

Prisma con base poligonale. Parametri: poly (array di punti 2D, almeno 3), y [y0,y1] (estensione lungo l'asse).

```
{"shape": "prism", "poly": [[0,0],[10,0],[10,8],[0,8]], "y": [0, 15]}
```

Nota sviluppatore: solo create.js.

6. Proprietà indirizzabili nelle espressioni (.pr, .z, .m, .ox, .oy, .oz, …)

Nelle stringhe-espressione si possono leggere proprietà già calcolate di ingranaggi e assi. Sono la base per parametrizzare distanze, moduli e posizioni in funzione della geometria reale.

6.1 Proprietà degli ingranaggi

Sintassi: `gears.<id>.<prop>` oppure `gears["<id>"].<prop>`. L'ingranaggio deve essere attivo nella variante corrente.

- **.z** — numero di denti (valore numerico di teeth dopo la valutazione).
- **.m** — modulo in mm (valore numerico di module).
- **.pr** — raggio primitivo in mm (di solito z·m/2; per alcuni tipi è calcolato diversamente).
- **.d** — diametro primitivo in mm (sempre 2 × .pr).

Esempi: `"gears.gear_P.pr + gears.gear_Q.pr"`, `"e * 2 / (gears.gear_O.z + gears.gear_S.z)"`.

6.2 Proprietà degli assi (origine e direzione)

Sintassi: `axes.<id>.<prop>` oppure `axes["<id>"].<prop>`. L'asse deve essere già posizionato (altrimenti si ottiene un errore NotPlaced).

- **.ox** — coordinata X dell'origine dell'asse (dopo il posizionamento).
- **.oy** — coordinata Y dell'origine dell'asse.
- **.oz** — coordinata Z dell'origine dell'asse.
- **.dx** — componente X del versore direzione dell'asse.
- **.dy** — componente Y del versore direzione.

- **.dz** — componente Z del versore direzione.

Queste proprietà esistono ancora nella versione attuale del simulatore (non sono state rimosse). Servono quando una posizione o una lunghezza dipende da dove è finito un altro asse dopo i vincoli di place/through.

Esempio: `"at": ["axes.axis_A.ox", "axes.axis_A.oy + 20", "axes.axis_A.oz"]`.

6.3 Altri riferimenti utili nelle espressioni

- Id degli slider definiti in `controls` (es. `latitude`, `zR`).
- Costanti definite in `consts` (nell'ordine in cui compaiono).
- Costante `pi` per π.
- Funzioni: sin, cos, tan, atan, atan2, sqrt, abs, min, max.

7. Posizione assiale dell'ingranaggio: w oppure pos + thickness

Ogni ingranaggio deve avere un intervallo lungo il proprio asse che indica dove sta il corpo del pezzo. Ci sono **due modi equivalenti**; non è obbligatorio usare proprio il campo "w".

7.1 Modo A — campo w

`"w": [w0, w1]` — due valori (numeri o espressioni) che sono le coordinate delle due facce del pezzo lungo l'asse (origine e verso = quelli dell'asse). L'ordine non deve essere crescente: il simulatore prende min e max.

```
{ "id": "g1", "type": "spur", "axis": "ax1",
  "teeth": 20, "module": 2, "w": [0, 10] }
```

7.2 Modo B — pos + thickness (insieme)

• **pos** = coordinata della faccia più vicina all'origine dell'asse.
• **thickness** = spessore; se positivo il pezzo si estende verso +asse, se negativo verso −asse.
I due campi devono comparire **insieme**. Il simulatore li converte internamente in un w (proprietà non enumerabile, non finisce in un eventuale export JSON).

```
{ "id": "g1", "type": "spur", "axis": "ax1",
  "teeth": 20, "module": 2, "pos": 0, "thickness": 10 }
```

Se nello stesso oggetto ci sono sia `w` sia `pos`/`thickness`, viene usato **w** e viene emesso un avviso. Se c'è solo uno tra pos e thickness, è un errore.

8. Foro centrale, dente di fissaggio (profilo a D) e asse abbinato

Per la stampa 3D e per il montaggio reale conviene che l'ingranaggio solidale all'asse abbia un foro con un lato piano ("a D"), e che l'albero abbia la stessa sezione. Il simulatore lo gestisce in modo automatico e parametrico.

8.1 Comportamento automatico del foro (bore)

Se nel gear non si scrive nulla su bore, il sistema cerca sulle parti cilindriche dell'asse (o dell'asse ospite se l'ingranaggio è free) quelle **centrate** (at assente o [0,0]) che intersecano lo spessore del gear, e prende il raggio massimo tra quelle.

• Ingranaggio **solidale** (default, fixed diverso da false e free diverso da true): foro a D con raggio = raggio dell'albero e lato piano (flat di default 0,8).
• Ingranaggio **libero** (free: true oppure fixed: false): foro **circolare** con raggio = 1,05 × raggio albero (gioco per farlo girare).

Si può forzare il comportamento:
• `"bore": false` — nessun foro.
• `"bore": 5` oppure `"bore": "expr"` — raggio esplicito (mm).
• `"flat": 0.8` — distanza del lato piano dal centro, in frazione del raggio del foro (0 < flat < 1). Default 0,8 se l'ingranaggio è solidale; `false` forza il foro circolare anche se solidale.
• `"flatAngle": 0` — orientamento del lato piano in **gradi** nel riferimento dell'asse (default 0). Tutti i gear solidali sullo stesso asse con lo stesso flatAngle hanno il piano allineato.

8.2 Albero con dente / profilo a D (parte cylinder dell'asse)

Sulla parte `cylinder` dell'asse si può specificare `"flat": { … }` per ottenere la sezione a D (il "dente" o chiavetta integrata):

• **depth** — profondità del piano rispetto al raggio esterno (mm): il piano sta a distanza r – depth dal centro.
• **dist** — alternativa a depth: distanza assoluta del piano dal centro (mm).
• **angle** oppure **ang** — orientamento del piano in gradi (stesso significato di flatAngle del gear).

Per far combaciare albero e foro: usare lo stesso angle/flatAngle e dimensionare depth/dist in modo coerente con il flat del gear (flat del gear è una frazione del raggio del foro; sul cylinder si lavora in mm).

```
// Asse con albero a D (raggio 4 mm, piano a 3.2 mm dal centro = depth 0.8)
{ "id": "ax1", "dir": [0,1,0], "ref": [1,0,0], "place": {"at":[0,0,0]},
  "parts": [
    { "shape": "cylinder", "r": 4, "w": [-10, 80], "n": 32,
      "flat": { "depth": 0.8, "angle": 0 } }
  ]
}

// Ingranaggio solidale: foro automatico a D (oppure esplicito)
{ "id": "g1", "type": "spur", "axis": "ax1",
  "teeth": 30, "module": 2, "w": [20, 30],
  "flat": 0.8, "flatAngle": 0 }

// Ingranaggio folle sullo stesso albero: foro circolare con gioco
{ "id": "g2", "type": "spur", "axis": "ax1",
  "teeth": 20, "module": 2, "w": [40, 50],
  "free": true }
```

Nota: le parti cylinder con `at` non nullo (perni, marker, ecc.) non vengono considerate albero per il calcolo del bore: contano solo i cilindri centrati sull'asse.

9. Marker degli ingranaggi: cosa sono, a cosa servono, come si specificano

Il marker è una piccola sfera attaccata all'ingranaggio (in realtà al frame dell'asse, virtuale se l'ingranaggio è free). Ruota insieme all'ingranaggio e serve a **leggere visivamente la rotazione** e a orientarsi nella scena (soprattutto quando i denti non si distinguono bene).

Per requisito del simulatore, **ogni ingranaggio ha un marker** se non si disattiva esplicitamente. La definizione sta **nell'oggetto gear** (campo marker), non nell'asse.

9.1 Valori ammessi di marker

• `omesso` oppure `true` — marker di default, raggio 4 mm, colore del materiale marker di scena, posizione automatica.
• `false` — nessun marker per questo ingranaggio.
• `numero` (es. 6) — raggio della sfera in mm; colore e posizione di default.
• `oggetto` — controllo completo:

```
"marker": {
  "radius": 5,          // oppure "r" / "size"; default 4
  "color": "#ff0000",   // colore della sfera; se omesso usa il materiale marker di scena
  "at": [x, y, z]       // posizione nel sistema locale dell'ingranaggio (espressioni ammesse)
}
```

Posizione automatica (se at è omesso): nel piano dell'ingranaggio, a circa 0,65 × raggio primitivo dall'asse, a metà spessore (oppure markerZ del tipo se definito). Per le ruote coniche si tiene conto di side e apexW.

9.2 Marker sulle parti dell'asse (colore speciale)

Sulle parti di un asse si può usare `"color": "marker"` (stringa letterale). La parte viene disegnata con lo stesso materiale dei marker degli ingranaggi, utile per riferimenti visivi fissi sull'albero (tacche, spine colorate).

```
{ "shape": "cylinder", "r": 0.5, "w": [-1, 11],
  "at": [".6*gears.gear_b.pr", 0], "n": 16, "color": "marker" }
```

9.3 Esempi pratici di marker sul gear

```
// Default (marker 4 mm, posizione automatica)
{ "id": "g1", "type": "spur", "axis": "ax1", "teeth": 20, "module": 2, "w": [0, 8] }

// Marker più grande e rosso, posizione automatica
{ "id": "g2", "type": "spur", "axis": "ax1", "teeth": 20, "module": 2, "w": [0, 8],
  "marker": { "radius": 6, "color": "#ff3333" } }

// Marker disattivato
{ "id": "g3", "type": "spur", "axis": "ax1", "teeth": 20, "module": 2, "w": [0, 8],
  "marker": false }

// Marker in posizione esplicita (sistema locale gear: x radiale, z lungo asse)
{ "id": "g4", "type": "spur", "axis": "ax1", "teeth": 20, "module": 2, "w": [0, 8],
  "marker": { "radius": 3, "color": "#00aa00", "at": [12, 0, 4] } }
```

10. Note generali sulle espressioni e formule

Quasi tutti i valori numerici accettano stringhe-espressione. Il valutatore non è un eval JavaScript grezzo: parsa solo espressioni aritmetiche con le funzioni e i riferimenti ammessi. L'ordine di valutazione rispetta le dipendenze (costanti → assi posizionati → ingranaggi, ecc.).

Operatori: + – * / e parentesi. Funzioni: sin, cos, tan, atan, atan2, sqrt, abs, min, max. Costante pi.

Esempi: `"yA1": "yA0 + face"`, `"e": "gears.gear_P.pr + gears.gear_Q.pr"`, `"mOS": "e * 2 / (gears.gear_O.z + gears.gear_S.z)"`, `"at": ["axes.ax1.ox", 0, "axes.ax1.oz + 10"]`.

11. Note per lo sviluppatore (riepilogo)

Per la grande maggioranza dei tipi è sufficiente modificare i file `create.js` e/o `solve.js` della cartella del tipo. Il nucleo (core/) non deve essere toccato per aggiungere o variare un tipo.

Casi particolari segnalati:

- **spur**: piccolo supporto angle/phase in core/09-resolve.js.
- **worm**: la formula del raggio è stata spostata nel tipo (non più globale).
- **hooke**: bug storico sul parametro arm (vedi README e commento in create.js).
- **scappamenti**: coesistono versioni classiche e _CS; verificare quale viene usata dalla macchina.
- **maltese / geneva**: la cinematica è concentrata nel link; i gear sono prevalentemente geometrici.
- Foro (bore) e marker: logica in core/09-resolve.js (computeBore) e core/11-scene.js; il profilo a D dell'albero è in types/parts/cylinder/create.js.
- Validazione generica (id, riferimenti, place, w/pos) in core/06-validate.js; proprietà .ox/.oy/.oz/.pr/.z/.m in core/02-expression.js.

Per aggiungere un tipo nuovo: copiare types/_template/gear o types/_template/link, rinominare NOME, implementare le funzioni necessarie e usare "type": "nome" nel JSON. Nessun elenco va aggiornato a mano.

Repository: https://github.com/jumpjack/mechsim
Documento generato per la versione V. 0.4 del simulatore (9 novembre 2026).